

Final Project document

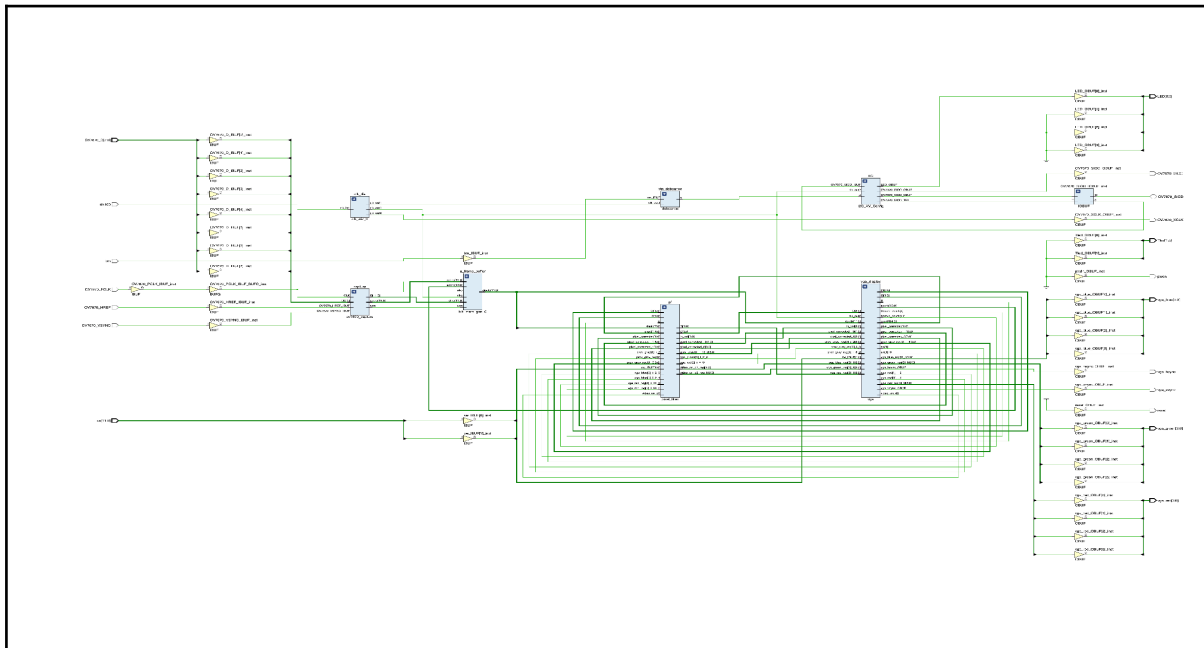
Final Project document	1
Group Member	1
Overall Design Block Diagram	3
Overall	3
Module Interactions & System Flow	3
Data Transfer Frequency	4
Design Decision	5
1. Module Architecture	5
2. Communication Protocols	6
3. Clock and Data Transfer Rate	7
4. Resource Utilization	8
Implementation Detail	10
Module : ov7670_top (ov7670_top.v)	10
Module : ov7670_capture (camera_capture.v)	14
Module : vga (vga_disp.v)	17
Module : pixel_filter (pixel_filter.v)	21
Module : I2C_AV_Config (camera_config.v)	25
Module : I2C_Controller (sccb_sender.v)	30
Module : debounce (debounce.v)	41
Challenge Faced	43
1. Edge Visual Noise and Garbage Pixels	43
2. Clock Domain Crossing (CDC) Issues	43
3. Severe BRAM Resource Constraints	43
4. Hardware Signal Integrity over Physical Wiring	44
Simulation & Testbenches	45
1. tb_vga.v	45
2. tb_pixel_filter.v	48
i. Normal Mode (sw = 0 / 2'b00):	50
ii. Grayscale Filter (sw = 1 / 2'b01):	50
iii. Color Splash - Blue Focus (sw = 2 / 2'b10):	50
iv. 1D Edge Detection (sw = 3 / 2'b11):	50

Group Member

Group name: “ [02] :) ”

Name	Student Number
Natthakorn Yiengyongphan	6731315721
Poonnawit Supawasuwat	6731332321
Sukrit Kankrasang	6731362121
Aut Suttitanamongkol	6731365021

Overall Design Block Diagram



Original files:

<https://drive.google.com/file/d/1cDy9ELqyXz0Dc4Di9oc6MaK7DvJDh57k/view?usp=sharing> →  schematic.pdf

Overall

The system is designed with a modular architecture to handle the high-speed data flow from the OV7670 camera to a VGA display. The following diagram illustrates the abstraction of the hardware logic, where each block represents a specific Verilog module.

Module Interactions & System Flow

The modules interact in a pipelined fashion to ensure real-time performance without frame tearing:

- **Clocking Subsystem (Clock Wizard):** This is the heart of the system's timing. It takes the 100MHz master clock and generates three synchronized clock domains: 50MHz for general logic/debouncing, 25MHz for the VGA controller and RAM read port, and 24MHz (XCLK) to drive the external camera sensor.
- **Camera Configuration (I2C_AV_Config & SCCB_Sender):** Upon power-up, the configuration master iterates through a Look-Up Table

(LUT) of 193 registers. It uses the SCCB (Serial Camera Control Bus) protocol to set the OV7670 into RGB565 mode and QVGA resolution.

- **Data Capture (ov7670_capture):** This module monitors the camera's VSYNC and HREF. When data is valid, it latches two 8-bit bytes to form a 16-bit RGB565 pixel, converts it to a 12-bit RGB444 format, and calculates the specific memory address for that pixel in the frame buffer.
- **Storage Subsystem (Block RAM):** Acts as a bridge between the camera and display domains. It uses a Simple Dual-Port configuration. The camera writes to Port A using the camera's PCLK, while the VGA controller reads from Port B using the 25MHz system clock.
- **Processing Pipeline (pixel_filter):** As pixels are read from RAM, they pass through this hardware engine. It applies color correction, gamma adjustment, or special effects (Grayscale, Edge Detection, Color Splash) based on the physical switch settings on the Basys 3 board.
- **Display Subsystem (vga_disp):** Generates the standard 640x480 timing. It implements a digital zoom/scaling algorithm to map the 320x240 image captured in memory to the full VGA screen resolution.

Data Transfer Frequency

Managing multiple clock domains is critical for stability.

Data Path / Domain	Clock Source	Frequency	Description
System Master	Onboard Oscillator	100 MHz	Primary input for the Clock Wizard.
Camera Drive (XCLK)	Clock Wizard	24 MHz	Sent to OV7670 to drive its internal logic.
Configuration (SCCB)	Internal Divider	~10 KHz	Slow serial clock for stable register writes.
Pixel Capture (PCLK)	Camera Sensor	~24 MHz	Frequency at which pixels arrive from the camera.
VGA Interface	Clock Wizard	25 MHz	Standard pixel clock for 640x480 @ 60Hz.
BRAM Write Access	PCLK	~24 MHz	Synchronous with incoming camera data.
BRAM Read Access	VGA Clock	25 MHz	Synchronous with display refresh timing.

Design Decision

1. Module Architecture

- Why did you structure your design this way?
- What are the advantages of using this approach? (e.g., modularity, efficiency, ease of debugging)

Design Decision: Modular System Architecture

Why did you structure your design this way?

- We adopted a top-down, modular architecture. The design is explicitly separated into distinct functional blocks: Camera Configuration (`camera_config`), Data Capture (`camera_capture`), Memory Storage (`blk_mem_gen_0`), Image Processing (`pixel_filter`), and Display Controller (`vga_disp`), all integrated under a single top-level module (`ov7670_top`).
- We structured the design this way primarily to separate the asynchronous clock domains. The camera operates on its own incoming Pixel Clock (PCLK at ~24MHz), while the display requires a strict 25MHz VGA clock. By dividing the system into independent modules, we can safely use a Block RAM (BRAM) as a synchronization bridge between the write-domain (Capture) and read-domain (VGA).

What are the advantages of using this approach?

1. Ease of Debugging: By isolating components, we could independently simulate and test specific parts of the system. For example, we could verify the VGA sync signals and screen alignment using a color test pattern before the camera hardware was fully integrated.
2. High Modularity: The image processing logic is completely isolated inside the `pixel_filter` module. This allowed us to easily add or modify new features (like Grayscale, Edge Detection, or Color Splash) without risking the stability of the sensitive camera capture or VGA timing logic.
3. Efficiency in Clock Management: Separating the read and write logic into different modules prevents timing violations (metastability) when transferring high-speed pixel data between the camera and the monitor.

2. Communication Protocols

- Why did you choose this specific protocol (e.g., SPI, I2C, UART, AXI) for communication?
- How does this protocol suit your design requirements?

Design Decision: Communication Protocol (SCCB / I2C)

Why did you choose this specific protocol for communication?

- For the communication between the FPGA and the OV7670 camera sensor, we implemented the SCCB (Serial Camera Control Bus) protocol, which is OmniVision's proprietary protocol that is heavily based on and highly compatible with the standard I2C protocol. We did not use AXI, SPI, or UART for this specific data path.

How does this protocol suit your design requirements?

1. Native Hardware Requirement: The OV7670 image sensor strictly mandates the use of SCCB to read and write its internal configuration registers. We utilized this protocol to successfully initialize the camera into RGB565 color mode and QVGA resolution at system startup.
2. Pin Efficiency (Resource Saving): SCCB is a 2-wire interface requiring only a serial clock line (SIOC) and a bidirectional serial data line (SIOD). This minimal pin footprint frees up the limited physical I/O pins on the Basys 3 board, which are heavily needed for the 8-bit parallel pixel data and VGA color outputs.
3. Appropriate Bandwidth and Timing for Initialization: The configuration process involves sending a sequence of 193 register cycles from a Look-Up Table (LUT). While only the first 165 instructions contain actual camera settings, the remaining cycles intentionally send dummy data to introduce a slight delay, allowing the sensor hardware to stabilize. Because this setup only happens once when the system powers on (or resets), high-speed data transfer is not necessary. Operating the I2C/SCCB controller at a low frequency (~10 KHz) ensures stable and highly reliable transmission without overcomplicating the logic design or wasting FPGA resources, which would be the case if we tried to implement a heavy protocol like AXI for a simple initialization task..

3. Clock and Data Transfer Rate

- Why did you select this specific clock frequency?
- How does the data rate impact system performance?
- Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?

Design Decision: Clock Frequencies and Data Transfer Rates

Why did you select this specific clock frequency?

Our system operates across multiple carefully selected clock domains generated by the FPGA's internal Clock Wizard (MMCM/PLL).

- 25 MHz for VGA: The industry standard pixel clock for a 640x480 @ 60Hz display is 25.175 MHz. Generating exactly 25 MHz is highly efficient on the FPGA and is well within the tolerance limits of modern VGA monitors to achieve a stable sync.
- 24 MHz for Camera (XCLK): The OV7670 sensor requires a master clock (XCLK) to drive its internal operations. 24 MHz is the manufacturer's recommended typical operating frequency, providing a good balance between a smooth frame rate and manageable pixel data output (PCLK).
- 10 KHz for SCCB: A very low frequency was chosen for the initialization protocol to ensure high reliability when configuring the camera registers.

How does the data rate impact system performance?

- The data transfer rate directly dictates our memory bandwidth and visual performance. By configuring the camera to output at QVGA resolution and downsampling the incoming 16-bit RGB565 data into 12-bit RGB444 before writing to the BRAM, we significantly reduced the required memory bandwidth. Because our VGA read clock (25 MHz) is slightly faster than the camera's pixel write clock (~24 MHz), the display controller can comfortably fetch pixel data without causing a system bottleneck, ensuring a continuous, flicker-free real-time video stream.

Did you consider constraints such as FPGA timing, external device compatibility, or noise margins?

Yes, several hardware constraints heavily influenced this design:

1. External Device Compatibility: We strictly adhered to the 25MHz

- VGA standard and the 24MHz OV7670 datasheet requirements to ensure the external hardware would sync correctly with the Basys 3 board.
2. **FPGA Timing & Metastability:** Because the camera writes data asynchronously to the VGA reading it, we utilized a Simple Dual-Port Block RAM as an elastic buffer. This safely crosses the clock domains without violating setup/hold times (timing constraints)
 3. **Noise Margins & Signal Integrity:** Connecting the FPGA to the camera via physical jumper wires introduces capacitance and electromagnetic interference (EMI). Sending a 100MHz clock directly over jumper wires would cause severe signal degradation. By stepping down the external clock to 24MHz and using a 10 KHz clock for the SCCB bus, we maintained high signal integrity and sufficient noise margins, preventing data corruption across the physical wires.

4. Resource Utilization

- How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?
- Why did you optimize certain parts of the design?

Design Decision: Resource Utilization and Performance Balance

How does your design balance resource usage (LUTs, FFs, BRAMs) and performance?

In video processing on FPGAs, the most critical bottleneck is usually memory (BRAM). The Basys 3 board has a limited BRAM capacity (approx. 1.8 Mbit). To balance this hardware limitation with the requirement of a real-time 640×480 @ 60Hz VGA display, we designed the architecture to strictly control resource consumption:

- **BRAM Efficiency:** We chose a baseline capture resolution of 320×240 rather than full VGA. This reduces the total number of pixels per frame by 75%. Furthermore, we downsampled the camera's 16-bit RGB565 output into a 12-bit RGB444 format before writing to memory. This perfectly matches the Basys 3's 12-bit VGA resistor DAC (4 bits per color) and ensures the entire frame buffer fits seamlessly within the available BRAM tiles without degrading the visible output quality.
- **LUTs and FFs Balance:** For the image processing pipeline, we

avoided heavy mathematical operations (like division or floating-point math). Instead, we used simple bit-shifts, look-up tables (for Gamma Correction), and basic weighted addition (for Grayscale: $Y = 0.3R + 0.6G + 0.1B$ approximated as $(R*3 + G*6 + B*1)$). This approach keeps LUT and Flip-Flop (FF) utilization extremely low while maintaining real-time processing speeds.

Why did you optimize certain parts of the design?

We actively optimized several specific parts of the pipeline to prevent timing violations and avoid running out of hardware resources:

- Optimization 1: Digital Zoom (Pixel Doubling): Instead of storing a 640×480 image (which would exceed the Basys 3 memory limits), we optimized the display logic by shifting the VGA coordinate counters by 1 bit ($HCnt \gg 1$, $VCnt \gg 1$). This allows us to digitally stretch the 320×240 BRAM image to fill the screen in real-time, requiring exactly zero extra memory.
- Optimization 2: 1D Edge Detection: Traditional edge detection algorithms (like the Sobel filter) require 2D spatial convolution and multiple "Line Buffers," which consume massive amounts of BRAM and LUTs. We optimized this by implementing a fast 1D edge detector. By storing only the previous pixel's grayscale value in a single register (`prev_gray`) and calculating the absolute difference with the current pixel, we achieved a visually effective edge-detection filter using almost zero additional resources.

Implementation Detail

Module : ov7670_top (ov7670_top.v)

Module code :

```
`timescale 1ns / 1ps

/////////////////////////////////////////////////////////////////
/
// Project: Real-Time Video Capture and Processing System
// Module Name: ov7670_top
// Description: Top-level module for interfacing OV7670 camera with VGA display.
//              Handles clock generation, camera configuration, image capture,
//              buffering in BRAM, image filtering, and VGA output.
/////////////////////////////////////////////////////////////////
/

module ov7670_top(
    // System Clock Input
    input  clk100,          // มาสเตอร์คล็อก 100MHz จากบอร์ด FPGA

    // OV7670 Camera Interface Signals
    input  OV7670_VSYNC,    // สัญญาณ Vertical Sync จากกล้อง
    input  OV7670_HREF,     // สัญญาณ Horizontal Reference จากกล้อง
    input  OV7670_PCLK,     // Pixel Clock จากกล้อง สำหรับซิงโครไนซ์ข้อมูลพิกเซล
    output OV7670_XCLK,     // System Clock ที่ส่งไปเลี้ยงตัวกล้อง (ทำงานที่ 24MHz)
    output OV7670_SIOC,     // SCCB Clock (I2C interface) สำหรับตั้งค่ากล้อง
    inout  OV7670_SIOD,     // SCCB Data (I2C interface) สำหรับตั้งค่ากล้อง
    input  [7:0] OV7670_D,  // ข้อมูลพิกเซลขนาน 8 บิตจากกล้อง

    // Output Signals
    output [3:0] LED,       // ไฟ LED แสดงสถานะ (LED[0] ติดเมื่อตั้งค่ากล้องเสร็จ)
    output [3:0] vga_red,   // สัญญาณสีแดง VGA 4 บิต
    output [3:0] vga_green, // สัญญาณสีเขียว VGA 4 บิต
    output [3:0] vga_blue,  // สัญญาณสีน้ำเงิน VGA 4 บิต
    output vga_hsync,       // สัญญาณซิงค์แนวนอนสำหรับจอภาพ VGA
    output vga_vsync,       // สัญญาณซิงค์แนวตั้งสำหรับจอภาพ VGA

    // Control Inputs
    input btn,              // ปุ่มกดสำหรับสั่ง Resend การตั้งค่ากล้อง
    output pwn,             // สัญญาณ Power Down ของกล้อง (Set เป็น 0 สำหรับการทำงานปกติ)
    output reset,           // สัญญาณ Reset ของกล้อง (Set เป็น 1)
```

```

input [11:0] sw,          // สวิตช์ควบคุมฟิลเตอร์และการแสดงผล
output [1:0] Tled         // ไฟสถานะเพิ่มเติม (ปัจจุบันไม่ได้ใช้งาน)
);

// Internal Wires and Signals
wire [16:0] frame_addr;    // แอดเดรสสำหรับอ่านข้อมูลจาก Buffer ไปแสดงผล VGA
wire [16:0] capture_addr;  // แอดเดรสสำหรับเขียนข้อมูลจากกล้องลง Buffer
wire capture_we;           // สัญญาณ Write Enable สำหรับเขียนข้อมูลลง RAM
wire config_finished;      // สถานะระบุว่าตั้งค่ารีจิสเตอร์กล้องเสร็จสิ้น
wire clk25;                // คล็อก 25MHz สำหรับระบบ VGA
wire clk50;                // คล็อก 50MHz สำหรับระบบ Debounce
wire clk;                  // คล็อกระบบทั่วไป
wire clk24;                // คล็อก 24MHz สำหรับ XCLK ของกล้อง
wire resend;               // สัญญาณสั่งเริ่มตั้งค่ากล้องใหม่หลังจาก Debounce
wire [11:0] frame_pixel;   // ข้อมูลพิกเซลที่อ่านออกมาจาก RAM
wire [11:0] data_16;       // ข้อมูลพิกเซล 12 บิตที่ได้จากการรวมข้อมูลจากกล้อง
wire [9:0] capture_x;      // ตำแหน่งพิกัด X ที่กำลังบันทึก
wire [9:0] capture_y;      // ตำแหน่งพิกัด Y ที่กำลังบันทึก

// Hardware Control Assignments
assign pwn = 0;            // 0: Normal mode, 1: Power down mode
assign reset = 1;          // Active High Reset (ปกติเป็น 1)
assign LED = {3'b0, config_finished}; // แสดงสถานะการตั้งค่าผ่าน LED
assign OV7670_XCLK = clk24; // จ่ายสัญญาณนาฬิกาให้กล้อง

// Button Debounce Module
// ป้องกันการสั้นของสัญญาณปุ่มกดเมื่อสั่ง Resend Configuration
debounce btn_debounce(
    .clk(clk50),
    .i(btn),
    .o(resend)
);

// Image Filtering Module
// รับพิกเซลดิบจาก RAM และประมวลผลตามค่า Switch
wire [11:0] filtered_pixel;
pixel_filter pf(
    .pixel_in(frame_pixel),
    .sw(sw),
    .pixel_out(filtered_pixel)
);

```

```
// VGA Controller Module
// สร้างสัญญาณ Sync และดึงข้อมูลภาพจาก Buffer ไปแสดงผลบนจอ
vga vga_display (
    .clk25      (clk25),
    .sw         (sw),
    .vga_red    (vga_red),
    .vga_green  (vga_green),
    .vga_blue   (vga_blue),
    .vga_hsync  (vga_hsync),
    .vga_vsync  (vga_vsync),
    .HCnt       (),
    .VCnt       (),
    .frame_addr (frame_addr),
    .frame_pixel (filtered_pixel)
);

// Block RAM Frame Buffer (Simple Dual-Port)
// เก็บข้อมูลภาพขนาด 320x240 พิกเซล
blk_mem_gen_0 u_frame_buffer(
    // Write Port (Camera Side)
    .clka  (OV7670_PCLK), // ใช้ PCLK ในการเขียน
    .wea   (capture_we),
    .addra (capture_addr),
    .dina  (data_16),

    // Read Port (VGA Side)
    .clkb  (clk25), // ใช้ 25MHz ในการอ่าน
    .addrb (frame_addr),
    .doutb (frame_pixel)
);

// Camera Capture Module
// แปลงสัญญาณข้อมูลจากกล้องเป็นพิกเซล 12 บิต และระบุแอดเดรสสำหรับจัดเก็บ
ov7670_capture capture(
    .pclk  (OV7670_PCLK),
    .vsync (OV7670_VSYNC),
    .href  (OV7670_HREF),
    .d      (OV7670_D),
    .addr  (capture_addr),
    .dout  (data_16),
    .we    (capture_we),
    .x     (capture_x),
```

```

        .y      (capture_y)
    );

    // Camera Configuration Module (SCCB)
    // ส่งข้อมูลตั้งค่ารีจิสเตอร์เริ่มต้นให้กล้อง OV7670
    I2C_AV_Config IIC(
        .iCLK      (clk25),
        .iRST_N    (!resend),
        .Config_Done (config_finished),
        .I2C_SDAT   (OV7670_SIOD),
        .I2C_SCLK   (OV7670_SIOC),
        .LUT_INDEX  (),
        .I2C_RDATA  ()
    );

    // Clock Wizard
    // สร้างสัญญาณนาฬิกาความถี่ต่างๆ จากคล็อก 100MHz ของบอร์ด
    clk_wiz_0 clk_div(
        .clk_in1    (clk100),
        .clk_out1   (clk50),      // 50MHz
        .clk_out2   (clk25),      // 25MHz
        .clk_out3   (clk),        // 100MHz
        .clk_out4   (clk24)       // 24MHz
    );

    // Default status for unused signals
    assign Tled[0] = 1'b0;
    assign Tled[1] = 1'b0;

endmodule

```

Module Description :

The top-level entity of the system that integrates all sub-modules. It manages clock distribution using a Clock Wizard, handles the interconnection between the camera interface and VGA display, and routes control signals from onboard switches and buttons to the internal logic.

Module : ov7670_capture (camera_capture.v)

Module code :

```

////////////////////////////////////
/
// Module Name: ov7670_capture
// Description: Module for capturing parallel pixel data from the OV7670 camera.
//              It latches two bytes to form a 16-bit RGB565 pixel, converts it
//              to RGB444 format, and generates memory addresses for a 320x240
//              buffer.
////////////////////////////////////
/

module ov7670_capture(
    // Clock and Synchronization Signals
    input pclk,                // Pixel Clock จากกล้อง ใช้เป็นสัญญาณนาฬิกาหลักของโมดูล
    [cite: 1245]
    input vsync,               // Vertical Sync ระบุการเริ่มต้นเฟรมใหม่ [cite: 1245]
    input href,               // Horizontal Reference ระบุช่วงที่มีข้อมูลพิกเซลที่ถูกต้องในแนวดิ่ง
    [cite: 1245]

    // Data Input
    input [7:0] d,            // ข้อมูลพิกเซลขนาน 8 บิตจากพอร์ตข้อมูลของกล้อง [cite: 1245]

    // Output Signals
    output [16:0] addr,       // แอดเดรสสำหรับเขียนลง RAM (คำนวณจากตำแหน่ง X, Y) [cite:
1245]
    output [11:0] dout,       // ข้อมูลพิกเซลในรูปแบบ RGB444 (12 บิต) [cite: 1245]
    output reg we,            // สัญญาณ Write Enable สำหรับสั่งบันทึกลง RAM [cite: 1245]

    // Position Debug/Status
    output [9:0] x,           // ตำแหน่งพิกเซลในแนวนอน (0-319) [cite: 1245]
    output [9:0] y            // ตำแหน่งพิกเซลในแนวดิ่ง (0-239) [cite: 1245]
);

// Internal Registers
reg [15:0] d_latch = 0;     // รีจิสเตอร์สำหรับพักข้อมูล 2 ไบต์ที่รับเข้ามา [cite: 1246]
reg [11:0] dout1 = 0;       // รีจิสเตอร์เก็บค่าพิกเซล RGB444 ที่พร้อมส่งออก [cite: 1246]
reg [1:0] wr_hold = 0;      // สัญญาณหน่วงสำหรับการควบคุมจังหวะการเขียน (Write Control)
[cite: 1246]

// Pixel Position Registers

```

```

reg [9:0] x_reg = 0;      // ตัวนับตำแหน่งพิกเซลในแนวนอน [cite: 1247]
reg [9:0] y_reg = 0;      // ตัวนับตำแหน่งพิกเซลในแนวตั้ง [cite: 1247]

// ===== RGB565 Extraction =====
// แยกส่วนประกอบสีจากข้อมูล 16 บิต (5 บิต แดง, 6 บิต เขียว, 5 บิต น้ำเงิน) [cite: 1248]
wire [4:0] r5 = d_latch[15:11];
wire [5:0] g6 = d_latch[10:5];
wire [4:0] b5 = d_latch[4:0];

// ===== RGB444 Conversion =====
// แปลงข้อมูลจาก 5/6 บิต ให้เหลือ 4 บิตต่อสีเพื่อลดขนาดการจัดเก็บ [cite: 1249]
wire [3:0] r4 = r5[4:1];
wire [3:0] g4 = g6[5:2];
wire [3:0] b4 = b5[4:1];

// Output Assignments
// คำนวณหา Linear Address สำหรับหน่วยความจำขนาด 320x240 [cite: 1250]
assign addr = y_reg * 320 + x_reg;
assign dout = dout1;

// Main Capture Logic
always @(posedge pclk) begin
    if (vsync) begin
        // เมื่อขึ้นเฟรมใหม่ ให้รีเซ็ตตำแหน่งและสถานะทั้งหมด [cite: 1251]
        x_reg <= 0;
        y_reg <= 0;
        wr_hold <= 0;
        we <= 0;
    end else begin
        // ทำการ Latch ข้อมูลทีละไบต์ต่อกันจนครบ 2 ไบต์เพื่อให้ได้ 1 พิกเซล [cite: 1252]
        d_latch <= {d_latch[7:0], d};

        // ตรวจสอบจังหวะข้อมูลพิกเซลที่ถูกต้อง (HREF)
        // wr_hold[1] จะเป็น High ทุกๆ 2 รอบของ pclk เมื่อได้รับข้อมูลครบ 1 พิกเซล [cite:
1253]
        wr_hold <= {wr_hold[0], (href && !wr_hold[0])};
        we <= wr_hold[1];

        if (wr_hold[1]) begin
            // บันทึกค่าสี RGB444 ลงในรีจิสเตอร์เตรียมส่งออก [cite: 1254]
            dout1 <= {r4, g4, b4};
        end
    end
end

```

```
// จัดการการเลื่อนตำแหน่งพิกัด X และ Y [cite: 1255]
if (x_reg < 319) begin
    x_reg <= x_reg + 1;
end else begin
    // เมื่อจบบรรทัด ให้กลับไปเริ่มต้น x ใหม่และเพิ่มค่า Y [cite: 1256]
    x_reg <= 0;
    if (y_reg < 239)
        y_reg <= y_reg + 1;
    else
        y_reg <= 0;
    end
end
end
end
end

// มอบหมายค่าตำแหน่งปัจจุบันไปยังพอร์ตเอาต์พุต [cite: 1260]
assign x = x_reg;
assign y = y_reg;

endmodule
```

Module Description :

This module is responsible for fetching raw 8-bit parallel data from the OV7670 camera. It synchronizes with the camera's Pixel Clock (PCLK) and synchronization signals (VSYNC, HREF) to reconstruct 16-bit RGB565 pixels, converts them to 12-bit RGB444 format, and generates write addresses for the frame buffer.

Module : vga (vga_disp.v)

Module code :

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
/
// Module Name: vga
// Description: VGA Controller module for 640x480 @ 60Hz display.
//              Includes digital zooming, image scaling (15/16 ratio),
//              alignment adjustment to avoid visual noise, and pipeline
//              synchronization with BRAM and Pixel Filter.
/////////////////////////////////////////////////////////////////
/

module vga(
    input clk25,          // สัญญาณนาฬิกา 25MHz สำหรับ VGA Timing
    input [1:0] sw,        // สวิตช์เลือกโหมดฟิลเตอร์ (ส่งต่อไปยัง pixel_filter)
    output reg [3:0] vga_red, // สัญญาณสีแดง VGA 4 บิต
    output reg [3:0] vga_green, // สัญญาณสีเขียว VGA 4 บิต
    output reg [3:0] vga_blue, // สัญญาณสีน้ำเงิน VGA 4 บิต
    output vga_hsync,      // สัญญาณ Horizontal Sync
    output vga_vsync,      // สัญญาณ Vertical Sync
    output [9:0] HCnt,      // ตำแหน่งพิกเซลแนวนอนปัจจุบัน (Horizontal Counter)
    output [9:0] VCnt,      // ตำแหน่งพิกเซลแนวตั้งปัจจุบัน (Vertical Counter)
    output [16:0] frame_addr, // แอดเดรสสำหรับอ่านข้อมูลพิกเซลจาก RAM
    input [11:0] frame_pixel // ข้อมูลพิกเซลจาก RAM (12-bit RGB444)
);

// =====
// 1. VGA Timing Parameters (640x480 @ 60Hz)
// นิยามค่าพารามิเตอร์มาตรฐานสำหรับการสร้างสัญญาณซิงค์
// =====

localparam H_DISPLAY = 640; // ส่วนแสดงผลแนวนอน
localparam H_L_BORDER = 48; // ขอบซ้าย (Back Porch)
localparam H_R_BORDER = 16; // ขอบขวา (Front Porch)
localparam H_RETRACE = 96; // ช่วงสัญญาณ Sync แนวนอน
localparam H_MAX = H_DISPLAY + H_L_BORDER + H_R_BORDER + H_RETRACE - 1;
localparam START_H_RETRACE = H_DISPLAY + H_R_BORDER;
localparam END_H_RETRACE = H_DISPLAY + H_R_BORDER + H_RETRACE - 1;

localparam V_DISPLAY = 480; // ส่วนแสดงผลแนวตั้ง
localparam V_T_BORDER = 10; // ขอบบน (Front Porch)

```

```

localparam V_B_BORDER = 33; // ขอบล่าง (Back Porch)
localparam V_RETRACE = 2; // ช่วงสัญญาณ Sync แนวตั้ง
localparam V_MAX = V_DISPLAY + V_T_BORDER + V_B_BORDER + V_RETRACE - 1;
localparam START_V_RETRACE = V_DISPLAY + V_B_BORDER;
localparam END_V_RETRACE = V_DISPLAY + V_B_BORDER + V_RETRACE - 1;

// Registers สำหรับตัวนับและสัญญาณ Sync
reg [9:0] h_count_reg = 0, h_count_next;
reg [9:0] v_count_reg = 0, v_count_next;
reg vsync_reg = 0, hsync_reg = 0;

// กระบวนการอัปเดตตัวนับและสร้างสัญญาณ Sync (Active Low)
always @(posedge clk25) begin
    v_count_reg <= v_count_next;
    h_count_reg <= h_count_next;
    vsync_reg <= ~(v_count_reg >= START_V_RETRACE && v_count_reg <=
END_V_RETRACE);
    hsync_reg <= ~(h_count_reg >= START_H_RETRACE && h_count_reg <=
END_H_RETRACE);
end

// ตรรกะการนับตำแหน่งพิกเซล (Next State Logic)
always @* begin
    h_count_next = (h_count_reg == H_MAX) ? 0 : h_count_reg + 1;
    v_count_next = (h_count_reg == H_MAX) ?
        ((v_count_reg == V_MAX) ? 0 : v_count_reg + 1) :
v_count_reg;
end

assign HCnt = h_count_reg;
assign VCnt = v_count_reg;

// ตรวจสอบว่าพิกัดปัจจุบันอยู่ในพื้นที่แสดงผลหรือไม่
wire video_on = (h_count_reg < H_DISPLAY) && (v_count_reg < V_DISPLAY);

// =====
// 2. UNIFORM DIGITAL ZOOM & Scaling
// ปรับสัดส่วนภาพจาก 320x240 ให้แสดงผลบน 640x480
// =====

// Pixel Doubling: หาค่าด้วย 2 (Shift Right 1) เพื่อขยายพิกเซลจาก 320 เป็น 640
wire [8:0] cam_x = h_count_reg >> 1; // ช่วง 0-319

```

```

wire [8:0] cam_y = v_count_reg >> 1; // ช่วง 0-239

// Scaling Logic (อัตราส่วน 15/16):
// เพื่อบีบสเกลภาพให้เล็กลงเล็กน้อยสำหรับพื้นที่ขอบเพื่อลดสัญญาณรบกวน (Garbage pixels)
wire [13:0] temp_x = cam_x * 15;
wire [13:0] temp_y = cam_y * 15;

// Image Alignment & Offset:
// safe_cam_x: ขยับแกน X ไปทางขวา 15 พิกเซล เพื่อหลบขยะสีจากขอบซ้ายของกล้อง
wire [8:0] safe_cam_x = 15 + (temp_x >> 4);
// safe_cam_y: ขยับแกน Y ลงมา 7 พิกเซล เพื่อให้ภาพอยู่กึ่งกลางจอพอดี
wire [8:0] safe_cam_y = 7 + (temp_y >> 4);

// ดึงข้อมูล Address สำหรับ RAM ด้วยพิกัดที่ปรับแต่งแล้ว
assign frame_addr = video_on ? ((safe_cam_y * 320) + safe_cam_x) : 0;

// =====
// 3. Pipeline Delay Management
// การหน่วงสัญญาณเพื่อให้ Sync กับข้อมูลที่อ่านมาจาก BRAM (ใช้เวลา 2 รอบคล็อก)
// =====
reg video_on_d1 = 0, video_on_d2 = 0;
reg hsync_d1 = 0, hsync_d2 = 0;
reg vsync_d1 = 0, vsync_d2 = 0;

always @(posedge clk25) begin
    video_on_d1 <= video_on;
    video_on_d2 <= video_on_d1;
    hsync_d1 <= hsync_reg;
    hsync_d2 <= hsync_d1;
    vsync_d1 <= vsync_reg;
    vsync_d2 <= vsync_d1;
end

// เอาต์พุตสัญญาณ Sync ที่ถูกหน่วงเวลาแล้ว
assign vga_hsync = hsync_d2;
assign vga_vsync = vsync_d2;

// =====
// 4. Pixel Filtering & Final Output
// ประมวลผลพิกเซลผ่าน Filter และส่งออกไปยังขา VGA
// =====
wire [11:0] filtered_pixel;

```

```
// เชื่อมต่อโมดูลตัวกรองภาพ (pixel_filter)
pixel_filter my_filter (
    .clk(clk25),
    .sw(sw),
    .pixel_in(frame_pixel),
    .pixel_out(filtered_pixel)
);

// กำหนดค่าสีให้กับ Output Registers (4 บิตต่อสี)
always @(posedge clk25) begin
    if (video_on_d2) begin
        vga_red    <= filtered_pixel[11:8];
        vga_green  <= filtered_pixel[7:4];
        vga_blue   <= filtered_pixel[3:0];
    end else begin
        // แสดงสีดำเมื่ออยู่นอกพื้นที่การแสดงผล (Blanking period)
        vga_red    <= 4'b0;
        vga_green  <= 4'b0;
        vga_blue   <= 4'b0;
    end
end
endmodule
```

Module Description :

The VGA Controller generates precise horizontal and vertical synchronization signals based on a 25MHz clock to drive a 640x480 @ 60Hz display. It includes digital scaling logic to upscale the 320x240 stored image to full-screen and manages pipeline delays to ensure pixel data from memory aligns perfectly with the sync signals.

Module : pixel_filter (pixel_filter.v)

Module code :

```

`timescale 1ns / 1ps
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/
// Module Name: pixel_filter
// Description: Real-time image processing module.
//              It performs color correction (RGB Boost/Reduction),
//              Gamma Correction, and implements four selectable filters:
//              Normal, Grayscale, Color Splash (Blue), and Edge Detection.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/

module pixel_filter(
    input clk,                // สัญญาณนาฬิกา (ใช้คล็อกเดียวกับระบบแสดงผล)
    input [1:0] sw,           // สวิตช์เลือกรูปแบบฟิลเตอร์ (2'b00 ถึง 2'b11)
    input [11:0] pixel_in,    // ข้อมูลพิกเซลขาเข้า RGB444 (12 บิต)
    output reg [11:0] pixel_out // ข้อมูลพิกเซลขาออกที่ผ่านการประมวลผลแล้ว
);

    // =====
    // 0. แยกสีดั้งเดิมจากกล้อง (Raw RGB Extraction)
    // แยกข้อมูล 12 บิตออกเป็นสามประกอบ แดง (R), เขียว (G), และน้ำเงิน (B) อย่างละ 4 บิต
    // =====
    wire [3:0] r_raw = pixel_in[11:8];
    wire [3:0] g_raw = pixel_in[7:4];
    wire [3:0] b_raw = pixel_in[3:0];

    // =====
    // 1. นำสีดั้งเดิมมาผ่าน Color LUT (Color Correction)
    // ปรับจูนค่าสีเพื่อชดเชยข้อบกพร่องของเซนเซอร์กล้อง OV7670
    // =====
    reg [3:0] r_clut;
    wire [3:0] g_clut;
    wire [3:0] b_clut;

    // แก้ไขสีแดง: เพิ่มความสว่าง (Boost) เล็กน้อยในช่วงสีกลาง
    always @(*) begin
        case (r_raw)
            4'h7: r_clut = 4'h8;
            4'h8: r_clut = 4'h9;

```

```

        4'h9: r_clut = 4'hA;
        default: r_clut = r_raw;
    endcase
end

// แก้ไขสีเขียว: ลดความอมเขียว (Green Tint) ซึ่งเป็นลักษณะเฉพาะของกล้องรุ่นนี้
assign g_clut = (g_raw > 4'h8) ? (g_raw - 1) : g_raw;

// แก้ไขสีน้ำเงิน: ปรับลดความเข้มและป้องกันค่าติดลบ (Underflow)
assign b_clut = (b_raw > 4'h0 && b_raw < 4'h6) ? (b_raw - 1) : b_raw;

// =====
// 2. สร้างฟังก์ชันสำหรับ Gamma LUT (Gamma Correction)
// ปรับปรุง Contrast และความสว่างของภาพให้เหมาะสมกับการแสดงผลบนจอภาพ
// =====
function [3:0] gamma_corr;
    input [3:0] in_val;
    begin
        case(in_val)
            4'h4: gamma_corr = 4'h5;
            4'h5: gamma_corr = 4'h6;
            4'h6: gamma_corr = 4'h7;
            4'h7: gamma_corr = 4'h9;
            4'h8: gamma_corr = 4'hA;
            4'h9: gamma_corr = 4'hB;
            4'hA: gamma_corr = 4'hC;
            4'hB: gamma_corr = 4'hD;
            4'hC: gamma_corr = 4'hE;
            4'hD: gamma_corr = 4'hF;
            4'hE: gamma_corr = 4'hF;
            4'hF: gamma_corr = 4'hF;
            default: gamma_corr = in_val; // สำหรับค่าความสว่างต่ำ (0-3) ให้คงค่าเดิม
        endcase
    end
endfunction

// นำค่าสีที่ผ่าน Color Correction แล้วมาเข้าสู่กระบวนการ Gamma Correction
wire [3:0] r = gamma_corr(r_clut);
wire [3:0] g = gamma_corr(g_clut);
wire [3:0] b = gamma_corr(b_clut);

// รวมสัญญาณ R, G, B เป็นพิกเซลที่ปรับปรุงสีเรียบร้อยแล้ว

```

```

wire [11:0] pixel_corrected = {r, g, b};

// =====
// 3. คำนวณฟิลเตอร์ต่างๆ (Image Processing Filters)
// =====

// 3.1 ฟิลเตอร์ภาพขาวดำ (Grayscale)
// ใช้สูตรน้ำหนักสีเพื่อให้ได้ความสว่างที่สมจริง (Y = 0.3R + 0.6G + 0.1B)
wire [7:0] gray8 = (r * 3) + (g * 6) + (b * 1);
wire [3:0] gray = gray8[7:4]; // นำบิตบนมาใช้งานเป็นค่าสีเทา 4 บิต

// 3.2 ฟิลเตอร์ตรวจจับขอบ (Edge Detection)
// เปรียบเทียบความแตกต่างของความสว่างพิกเซลปัจจุบันกับพิกเซลก่อนหน้า
reg [3:0] prev_gray = 0;
always @(posedge clk) begin
    prev_gray <= gray;
end
wire [4:0] diff = (gray > prev_gray) ? (gray - prev_gray) : (prev_gray -
gray);
// หากความต่างมากกว่า Threshold (ค่า 3) ให้แสดงเป็นสีขาว (ขอบ) มิฉะนั้นเป็นสีดำ
wire [3:0] edge_val = (diff >= 3) ? 4'hF : 4'h0;

// 3.3 ระบบดูดสี (Color Splash - Blue focus)
// ตรวจสอบเงื่อนไขว่าพิกเซลนี้เป็น "สีน้ำเงิน" หรือไม่ โดยเปรียบเทียบสัดส่วนสี B กับ R และ G
wire [4:0] r5 = {1'b0, r};
wire [4:0] g5 = {1'b0, g};
wire [4:0] b5 = {1'b0, b};
wire is_blue = (b5 > r5 + 2) && (b5 > g5 + 1) && (b5 > 2);

// =====
// 4. MUX: ตัวเลือกฟิลเตอร์เพื่อแสดงผลออกจอ (Output Multiplexer)
// เลือกแสดงภาพตามสถานะของสวิตช์ควบคุม
// =====
always @(*) begin
    case(sw)
        2'b00: pixel_out = pixel_corrected; // 00: ภาพสีปกติ
        2'b01: pixel_out = {gray, gray, gray}; // 01: ภาพขาวดำ
        2'b10: pixel_out = is_blue ? pixel_corrected : {gray, gray, gray}; //
        2'b11: pixel_out = {edge_val, edge_val, edge_val}; // 11: แสดง
    end
end

```

ที่ปรับปรุงสีแล้ว

ค่าสมบูรณ์

10: เฉพาะสีน้ำเงินที่เป็นสี ส่วนอื่นเป็นขาวดำ

```
เฉพาะเส้นขอบของวัตถุ  
    default: pixel_out = pixel_corrected;  
endcase  
end  
  
endmodule
```

Module Description :

A dedicated hardware image processing engine. It performs real-time color correction and provides multiple selectable visual effects, including Grayscale conversion, Edge Detection, and a Color Splash (Blue) filter. The active filter is selected dynamically via physical switches on the FPGA board.

Module : I2C_AV_Config (camera_config.v)

Module code :

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
/
// Module Name: I2C_AV_Config
// Description: Module for configuring the OV7670 camera registers via SCCB
// (I2C-like) protocol.
//           It iterates through a Look-Up Table (LUT) containing register
//           addresses
//           and configuration data to initialize the camera settings.
/////////////////////////////////////////////////////////////////
/

module I2C_AV_Config
(
    // Global Signals
    input          iCLK,          // สัญญาณนาฬิกาหลัก (25MHz)
    input          iRST_N,        // สัญญาณรีเซ็ตแบบ Active Low

    // I2C/SCCB Interface Side
    output          I2C_SCLK,      // สัญญาณนาฬิกา SCCB (I2C Clock)
    inout           I2C_SDAT,      // สายสัญญาณข้อมูล SCCB (I2C Data) แบบ
    Bi-directional

    // Status and Monitoring Signals
    output reg      Config_Done, // สถานะระบุว่าการตั้งค่ารีจิสเตอร์ทั้งหมดเสร็จสิ้นแล้ว
    output reg [7:0] LUT_INDEX,  // ดัชนีระบุตำแหน่งปัจจุบันใน Look-Up Table (LUT)
    output [7:0]    I2C_RDATA     // ข้อมูลที่อ่านได้จาก I2C (สำหรับกรณีตรวจสอบ ID)
);

// Parameter Definitions
parameter LUT_SIZE = 193;      // จำนวนรายการรีจิสเตอร์ทั้งหมดที่ต้องตั้งค่า

///////////////////////////////////////////////////////////////// I2C Control Clock Generation ///////////////////////////////////////////////////////////////////
// ทำการหารสัญญาณนาฬิกาจาก 25MHz เพื่อสร้างสัญญาณนาฬิกาสำหรับควบคุม I2C (10 KHz)
parameter CLK_Freq = 25_000000; // ความถี่คล็อกขาเข้า 25 MHz
parameter I2C_Freq = 10_000;     // ความถี่ I2C ที่ต้องการ 10 KHz
reg [15:0] mI2C_CLK_DIV;          // ตัวนับสำหรับหารความถี่ (Clock Divider)
reg        mI2C_CTRL_CLK;         // สัญญาณนาฬิกาควบคุมภายในสำหรับ I2C

```

```

always @(posedge iCLK or negedge iRST_N)
begin
    if(!iRST_N)
        begin
            mI2C_CLK_DIV    <=  0;
            mI2C_CTRL_CLK   <=  0;
        end
    else
        begin
            // นับจำนวนรอบตามอัตราส่วนความถี่
            if( mI2C_CLK_DIV < (CLK_Freq/I2C_Freq)/2)
                mI2C_CLK_DIV <= mI2C_CLK_DIV + 1'd1;
            else
                begin
                    mI2C_CLK_DIV <= 0;
                    mI2C_CTRL_CLK <= ~mI2C_CTRL_CLK; // ทำการ Toggle สัญญาณนาฬิกา
                end
            end
        end
    end

// Edge Detection for I2C Clock
// ตรวจสอบขอบขาของ mI2C_CTRL_CLK เพื่อใช้เป็นจังหวะในการเปลี่ยนข้อมูล
reg i2c_en_r0, i2c_en_r1;
always @(posedge iCLK or negedge iRST_N)
begin
    if(!iRST_N)
        begin
            i2c_en_r0 <= 0;
            i2c_en_r1 <= 0;
        end
    else
        begin
            i2c_en_r0 <= mI2C_CTRL_CLK;
            i2c_en_r1 <= i2c_en_r0;
        end
    end

    wire    i2c_negclk = (i2c_en_r1 & ~i2c_en_r0) ? 1'b1 : 1'b0; // สร้าง Pulse เมื่อ
เกิดขอบขา

////////// Configuration Control State Machine
//////////

```



```

        mI2C_GO      <= 0;
        if (~mI2C_ACK)           // หากได้รับสัญญาณ ACK (Active Low)
แสดงว่าส่งสำเร็จ

            mSetup_ST <= 2;      // ไปยังขั้นตอนถัดไป
        else
            mSetup_ST <= 0;      // หากไม่ได้รับ ACK ให้ทำการส่งซ้ำ
(Repeat Transfer)
        end
    end
2: begin                          // Next Index State: เพิ่มตำแหน่งดัชนี
LUT

    LUT_INDEX <= LUT_INDEX + 8'd1;
    mSetup_ST <= 0;
    mI2C_GO   <= 0;
    mI2C_WR   <= 0;
    end
endcase
end
else
    begin
        // เมื่อส่งครบทุกรายการใน LUT แล้ว ให้คงสถานะเสร็จสิ้นไว้
        Config_Done <= 1'b1;
        LUT_INDEX <= LUT_INDEX;
        mSetup_ST <= 0;
        mI2C_GO   <= 0;
        mI2C_WR   <= 0;
        end
    end
end

////////////////////////////////////
// Instance ของโมดูล Look-Up Table (LUT)
// ทำหน้าที่จัดเก็บและจ่ายข้อมูล Register Address และ Data ตามดัชนี (LUT_INDEX)
wire [15:0] LUT_DATA;
I2C_OV7670_RGB565_Config OV7670_RGB565_Config
(
    .LUT_INDEX      (LUT_INDEX),
    .LUT_DATA       (LUT_DATA)
);

////////////////////////////////////
// Instance ของโมดูล SCCB/I2C Controller

```

```
// ทำหน้าที่ส่งสัญญาณตามมาตรฐานโปรโตคอล I2C ไปยังขากล้องโดยตรง
I2C_Controller sccb_sender
(
    .iCLK          (iCLK),
    .iRST_N        (iRST_N),

    .I2C_CLK        (mI2C_CTRL_CLK),    // สัญญาณนาฬิกาควบคุม
    .I2C_EN         (i2c_negclk),        // สัญญาณ Enable ข้อมูล
    .I2C_WDATA       ({8'h42, LUT_DATA}), // ข้อมูลที่จะเขียน: [Address ของกล้อง 0x42,
ข้อมูลจาก LUT]
    .I2C_SCLK        (I2C_SCLK),          // ขาเอาต์พุตนาฬิกาจริง
    .I2C_SDAT        (I2C_SDAT),          // ขาข้อมูลจริง

    .GO             (mI2C_GO),             // สัญญาณสั่งทำงาน
    .WR             (mI2C_WR),             // ประเภทการทำงาน Write/Read
    .ACK            (mI2C_ACK),            // สัญญาณตอบรับจากปลายทาง
    .END            (mI2C_END),            // สถานะจบการส่ง 1 ชุดข้อมูล
    .I2C_RDATA       (I2C_RDATA)          // ข้อมูลที่อ่านได้
);

////////////////////////////////////

endmodule
```

Module Description :

This module acts as the configuration master for the OV7670 camera. It utilizes a state machine to iterate through a Look-Up Table (LUT) of register settings, sending them via the SCCB protocol to initialize the camera's operation mode, resolution, and color format.

Module : I2C_Controller (sccb_sender.v)

Module code :

```

`timescale 1ns / 1ps
/////////////////////////////////////////////////////////////////
/
// Module Name: I2C_Controller
// Description: Lower-level I2C/SCCB Master Controller.
//             Handles start/stop conditions, byte transmission,
//             and ACK/NACK bit processing for both Write and Read operations.
/////////////////////////////////////////////////////////////////
/

module I2C_Controller
(
    // Global Signals
    input          iCLK,          // สัญญาณนาฬิกาหลักของระบบ
    input          iRST_N,       // สัญญาณรีเซ็ตแบบ Active Low

    // I2C transfer control
    input          I2C_CLK,      // สัญญาณนาฬิกาสำหรับข้อมูล (Data Transfer Clock)
    input          I2C_EN,      // สัญญาณเปิดการทำงานข้อมูล (Data Enable)
    input [23:0]   I2C_WDATA,    // ข้อมูลที่จะส่ง [SLAVE_ADDR, SUB_ADDR, DATA]
    output         I2C_SCLK,     // ขาเอาต์พุตนาฬิกา I2C (SCL)
    inout         I2C_SDAT,     // ขาข้อมูล I2C แบบสองทิศทาง (SDA)
    input         WR,           // โหมดการทำงาน: 1 = Write, 0 = Read
    input         GO,           // สัญญาณเริ่มการรับ-ส่งข้อมูล (Start Trigger)
    output        ACK,          // สถานะการตอบรับ (Acknowledge Status)
    output reg     END,          // สถานะเสร็จสิ้นการรับ-ส่ง (Transfer End)
    output reg [7:0] I2C_RDATA   // ข้อมูลที่อ่านได้จากอุปกรณ์ (กรณีโหมด Read)
);

// Internal Registers and Wires
reg    I2C_BIT;               // บิตข้อมูลที่จะส่งออกไปยังสายสัญญาณ
reg    SCLK;                  // สถานะนาฬิกา I2C ภายใน
reg [5:0] SD_COUNTER;        // ตัวนับสถานะ (State Counter) สำหรับลำดับการส่งบิต

// I2C Clock Generation Logic
// กำหนดช่วงเวลาที่ I2C_SCLK จะทำงานตามจังหวะ I2C_CLK ในแต่ละสถานะของตัวนับ
wire    I2C_SCLK1 = (GO == 1 &&
                    ((SD_COUNTER >= 5 && SD_COUNTER <= 12 || SD_COUNTER == 14)

```

```

        (SD_COUNTER >= 16 && SD_COUNTER <= 23 || SD_COUNTER == 25)
||
        (SD_COUNTER >= 27 && SD_COUNTER <= 34 || SD_COUNTER ==
36))) ? I2C_CLK : SCLK;

    wire    I2C_SCLK2 = (GO == 1 &&
        ((SD_COUNTER >= 5 && SD_COUNTER <= 12 || SD_COUNTER == 14)
||
        (SD_COUNTER >= 16 && SD_COUNTER <= 23 || SD_COUNTER == 25)
||
        (SD_COUNTER >= 33 && SD_COUNTER <= 40 || SD_COUNTER == 42)
||
        (SD_COUNTER >= 45 && SD_COUNTER <= 52 || SD_COUNTER ==
54))) ? I2C_CLK : SCLK;

    assign  I2C_SCLK = WR ? I2C_SCLK1 : I2C_SCLK2;

    // SDA Output/Input Control (Tri-state Logic)
    // ระบุช่วงเวลาที่ขา SDAT จะเป็น Output (ส่งข้อมูล) หรือ High-Z (รอรับ ACK/Data)
    wire    SDO1 = ((SD_COUNTER == 13 || SD_COUNTER == 14) ||
        (SD_COUNTER == 24 || SD_COUNTER == 25) ||
        (SD_COUNTER == 35 || SD_COUNTER == 36)) ? 1'b0 : 1'b1;

    wire    SDO2 = ((SD_COUNTER == 13 || SD_COUNTER == 14) ||
        (SD_COUNTER == 24 || SD_COUNTER == 25) ||
        (SD_COUNTER == 41 || SD_COUNTER == 42) ||
        (SD_COUNTER >= 44 && SD_COUNTER <= 52)) ? 1'b0 : 1'b1;

    wire    SDO = WR ? SDO1 : SDO2;
    assign  I2C_SDAT = SDO ? I2C_BIT : 1'bz;

    // ACK Status Assignments
    reg     ACKW1, ACKW2, ACKW3; // ACK สำหรับโหมด Write
    reg     ACKR1, ACKR2, ACKR3; // ACK สำหรับโหมด Read
    assign  ACK = WR ? (ACKW1 | ACKW2 | ACKW3) : (ACKR1 | ACKR2 | ACKR3);

    // Counter Control Logic
    always @(posedge iCLK or negedge iRST_N)
    begin
        if (!iRST_N)
            SD_COUNTER <= 6'b0;
        else if(I2C_EN)

```

[illegible]

```

6'd10: I2C_BIT <= I2C_WDATA[17];
6'd11: I2C_BIT <= I2C_WDATA[16];
6'd12: I2C_BIT <= 0;           // High-Z สำหรับ ACK
6'd13: ACKW1 <= I2C_SDAT;     // อ่านค่า ACK1
6'd14: I2C_BIT <= 0;

// SUB ADDR (8 bits) + ACK2
6'd15: I2C_BIT <= I2C_WDATA[15];
6'd16: I2C_BIT <= I2C_WDATA[14];
6'd17: I2C_BIT <= I2C_WDATA[13];
6'd18: I2C_BIT <= I2C_WDATA[12];
6'd19: I2C_BIT <= I2C_WDATA[11];
6'd20: I2C_BIT <= I2C_WDATA[10];
6'd21: I2C_BIT <= I2C_WDATA[9];
6'd22: I2C_BIT <= I2C_WDATA[8];
6'd23: I2C_BIT <= 0;
6'd24: ACKW2 <= I2C_SDAT;     // อ่านค่า ACK2
6'd25: I2C_BIT <= 0;

// WRITE DATA (8 bits) + ACK3
6'd26: I2C_BIT <= I2C_WDATA[7];
6'd27: I2C_BIT <= I2C_WDATA[6];
6'd28: I2C_BIT <= I2C_WDATA[5];
6'd29: I2C_BIT <= I2C_WDATA[4];
6'd30: I2C_BIT <= I2C_WDATA[3];
6'd31: I2C_BIT <= I2C_WDATA[2];
6'd32: I2C_BIT <= I2C_WDATA[1];
6'd33: I2C_BIT <= I2C_WDATA[0];
6'd34: I2C_BIT <= 0;
6'd35: ACKW3 <= I2C_SDAT;     // อ่านค่า ACK3
6'd36: I2C_BIT <= 0;

// STOP Condition
6'd37: begin SCLK <= 0; I2C_BIT <= 0; end
6'd38: SCLK <= 1;
6'd39: begin I2C_BIT <= 1; END <= 1; end
default: begin I2C_BIT <= 1; SCLK <= 1; end
endcase
end

else // Case 2: I2C Read Sequence
begin
case(SD_COUNTER)

```

```

6'd0 : begin SCLK <= 1; I2C_BIT <= 1; ACKW1 <= 1; ACKR1 <= 1;
END <= 0; end

// START + SLAVE ADDR + SUB ADDR (Phase 1)
6'd1 : begin SCLK <= 1; I2C_BIT <= 1; END <= 0; end
6'd2 : I2C_BIT <= 0;
6'd3 : SCLK <= 0;
6'd4 : I2C_BIT <= I2C_WDATA[23];
6'd5 : I2C_BIT <= I2C_WDATA[22];
6'd6 : I2C_BIT <= I2C_WDATA[21];
6'd7 : I2C_BIT <= I2C_WDATA[20];
6'd8 : I2C_BIT <= I2C_WDATA[19];
6'd9 : I2C_BIT <= I2C_WDATA[18];
6'd10: I2C_BIT <= I2C_WDATA[17];
6'd11: I2C_BIT <= I2C_WDATA[16];
6'd12: I2C_BIT <= 0;
6'd13: ACKR1 <= I2C_SDAT;
6'd14: I2C_BIT <= 0;
6'd15: I2C_BIT <= I2C_WDATA[15];
6'd16: I2C_BIT <= I2C_WDATA[14];
6'd17: I2C_BIT <= I2C_WDATA[13];
6'd18: I2C_BIT <= I2C_WDATA[12];
6'd19: I2C_BIT <= I2C_WDATA[11];
6'd20: I2C_BIT <= I2C_WDATA[10];
6'd21: I2C_BIT <= I2C_WDATA[9];
6'd22: I2C_BIT <= I2C_WDATA[8];
6'd23: I2C_BIT <= 0;
6'd24: ACKR2 <= I2C_SDAT;
6'd25: I2C_BIT <= 0;

// RESTART Condition
6'd26: begin SCLK <= 0; I2C_BIT <= 0; end
6'd27: SCLK <= 1;
6'd28: begin I2C_BIT <= 1; end
6'd29: begin SCLK <= 1; I2C_BIT <= 1; end
6'd30: I2C_BIT <= 0;
6'd31: SCLK <= 0;

// SLAVE ADDR (Read) + READ DATA
6'd32: I2C_BIT <= I2C_WDATA[23];
6'd33: I2C_BIT <= I2C_WDATA[22];
6'd34: I2C_BIT <= I2C_WDATA[21];

```

```

        6'd35: I2C_BIT <= I2C_WDATA[20];
        6'd36: I2C_BIT <= I2C_WDATA[19];
        6'd37: I2C_BIT <= I2C_WDATA[18];
        6'd38: I2C_BIT <= I2C_WDATA[17];
        6'd39: I2C_BIT <= 1'b1;          // Read Flag
        6'd40: I2C_BIT <= 0;
        6'd41: ACKR3 <= I2C_SDAT;
        6'd42: I2C_BIT <= 0;
        6'd43: I2C_BIT <= 0;
        6'd44: I2C_BIT <= 0;

        // Capture Data bits from SDAT
        6'd45: I2C_RDATA[7] <= I2C_SDAT;
        6'd46: I2C_RDATA[6] <= I2C_SDAT;
        6'd47: I2C_RDATA[5] <= I2C_SDAT;
        6'd48: I2C_RDATA[4] <= I2C_SDAT;
        6'd49: I2C_RDATA[3] <= I2C_SDAT;
        6'd50: I2C_RDATA[2] <= I2C_SDAT;
        6'd51: I2C_RDATA[1] <= I2C_SDAT;
        6'd52: I2C_RDATA[0] <= I2C_SDAT;
        6'd53: I2C_BIT <= 1;          // NACK หลังอ่านเสร็จ
        6'd54: I2C_BIT <= 0;

        6'd55: begin SCLK <= 0; I2C_BIT <= 0; end
        6'd56: SCLK <= 1;
        6'd57: begin I2C_BIT <= 1; END <= 1; end
        default: begin I2C_BIT <= 1; SCLK <= 1; end
        endcase
    end

end

else
    begin
        SCLK <= 1;
        I2C_BIT <= 1;
        ACKW1 <= 1; ACKW2 <= 1; ACKW3 <= 1;
        ACKR1 <= 1; ACKR2 <= 1; ACKR3 <= 1;
        END <= 0;
        I2C_RDATA <= I2C_RDATA;
    end

end

end
endmodule

```

```

////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/
// Module Name: I2C_OV7670_RGB565_Config
// Description: Configuration Look-Up Table (LUT) for the OV7670 Camera.
//              Contains a series of register addresses and values to initialize
//              the camera in RGB565 mode with specific image properties.
////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////////
/

module I2C_OV7670_RGB565_Config
(
    input      [7:0]    LUT_INDEX, // ดัชนีระบุตำแหน่งในตาราง
    output reg [15:0]   LUT_DATA   // ข้อมูลรีจิสเตอร์ [Address, Value]
);

parameter Read_DATA    = 0;
parameter SET_OV7670  = 2;

always@(*)
begin
    case(LUT_INDEX)
        // Camera Configuration Registers:
        // Format: {Register_Address, Data_Value}
        SET_OV7670 + 0 : LUT_DATA    = 16'h1214;
        SET_OV7670 + 1 : LUT_DATA    = 16'h40d0;
        SET_OV7670 + 2 : LUT_DATA    = 16'h3a04;
        SET_OV7670 + 3 : LUT_DATA    = 16'h3dc8;
        SET_OV7670 + 4 : LUT_DATA    = 16'h1e31;
        SET_OV7670 + 5 : LUT_DATA    = 16'h6b00;
        SET_OV7670 + 6 : LUT_DATA    = 16'h32b6;
        SET_OV7670 + 7 : LUT_DATA    = 16'h1713;
        SET_OV7670 + 8 : LUT_DATA    = 16'h1801;
        SET_OV7670 + 9 : LUT_DATA    = 16'h1902;
        SET_OV7670 + 10 : LUT_DATA   = 16'h1a7a;
        SET_OV7670 + 11 : LUT_DATA   = 16'h030a;
        SET_OV7670 + 12 : LUT_DATA   = 16'h0c00;
        SET_OV7670 + 13 : LUT_DATA   = 16'h3e00;
        SET_OV7670 + 14 : LUT_DATA   = 16'h7000;
        SET_OV7670 + 15 : LUT_DATA   = 16'h7100;
        SET_OV7670 + 16 : LUT_DATA   = 16'h7211;
        SET_OV7670 + 17 : LUT_DATA   = 16'h7300;
    endcase
end

```

```
SET_OV7670 + 18 : LUT_DATA = 16'ha202;
SET_OV7670 + 19 : LUT_DATA = 16'h1180;
SET_OV7670 + 20 : LUT_DATA = 16'h7a20;
SET_OV7670 + 21 : LUT_DATA = 16'h7b1c;
SET_OV7670 + 22 : LUT_DATA = 16'h7c28;
SET_OV7670 + 23 : LUT_DATA = 16'h7d3c;
SET_OV7670 + 24 : LUT_DATA = 16'h7e55;
SET_OV7670 + 25 : LUT_DATA = 16'h7f68;
SET_OV7670 + 26 : LUT_DATA = 16'h8076;
SET_OV7670 + 27 : LUT_DATA = 16'h8180;
SET_OV7670 + 28 : LUT_DATA = 16'h8288;
SET_OV7670 + 29 : LUT_DATA = 16'h838f;
SET_OV7670 + 30 : LUT_DATA = 16'h8496;
SET_OV7670 + 31 : LUT_DATA = 16'h85a3;
SET_OV7670 + 32 : LUT_DATA = 16'h86af;
SET_OV7670 + 33 : LUT_DATA = 16'h87c4;
SET_OV7670 + 34 : LUT_DATA = 16'h88d7;
SET_OV7670 + 35 : LUT_DATA = 16'h89e8;
SET_OV7670 + 36 : LUT_DATA = 16'h13e0;
SET_OV7670 + 37 : LUT_DATA = 16'h0000;
SET_OV7670 + 38 : LUT_DATA = 16'h1000;
SET_OV7670 + 39 : LUT_DATA = 16'h0d00;
SET_OV7670 + 40 : LUT_DATA = 16'h1428;
SET_OV7670 + 41 : LUT_DATA = 16'ha505;
SET_OV7670 + 42 : LUT_DATA = 16'hab07;
SET_OV7670 + 43 : LUT_DATA = 16'h2475;
SET_OV7670 + 44 : LUT_DATA = 16'h2563;
SET_OV7670 + 45 : LUT_DATA = 16'h26a5;
SET_OV7670 + 46 : LUT_DATA = 16'h9f78;
SET_OV7670 + 47 : LUT_DATA = 16'ha068;
SET_OV7670 + 48 : LUT_DATA = 16'ha103;
SET_OV7670 + 49 : LUT_DATA = 16'ha6df;
SET_OV7670 + 50 : LUT_DATA = 16'ha7df;
SET_OV7670 + 51 : LUT_DATA = 16'ha8f0;
SET_OV7670 + 52 : LUT_DATA = 16'ha990;
SET_OV7670 + 53 : LUT_DATA = 16'haa94;
SET_OV7670 + 54 : LUT_DATA = 16'h13ef;
SET_OV7670 + 55 : LUT_DATA = 16'h0e61;
SET_OV7670 + 56 : LUT_DATA = 16'h0f4b;
SET_OV7670 + 57 : LUT_DATA = 16'h1602;
SET_OV7670 + 58 : LUT_DATA = 16'h2102;
SET_OV7670 + 59 : LUT_DATA = 16'h2291;
```

```
SET_OV7670 + 60 : LUT_DATA = 16'h2907;
SET_OV7670 + 61 : LUT_DATA = 16'h330b;
SET_OV7670 + 62 : LUT_DATA = 16'h350b;
SET_OV7670 + 63 : LUT_DATA = 16'h371d;
SET_OV7670 + 64 : LUT_DATA = 16'h3871;
SET_OV7670 + 65 : LUT_DATA = 16'h392a;
SET_OV7670 + 66 : LUT_DATA = 16'h3c78;
SET_OV7670 + 67 : LUT_DATA = 16'h4d40;
SET_OV7670 + 68 : LUT_DATA = 16'h4e20;
SET_OV7670 + 69 : LUT_DATA = 16'h6900;
SET_OV7670 + 70 : LUT_DATA = 16'h7419;
SET_OV7670 + 71 : LUT_DATA = 16'h8d4f;
SET_OV7670 + 72 : LUT_DATA = 16'h8e00;
SET_OV7670 + 73 : LUT_DATA = 16'h8f00;
SET_OV7670 + 74 : LUT_DATA = 16'h9000;
SET_OV7670 + 75 : LUT_DATA = 16'h9100;
SET_OV7670 + 76 : LUT_DATA = 16'h9200;
SET_OV7670 + 77 : LUT_DATA = 16'h9600;
SET_OV7670 + 78 : LUT_DATA = 16'h9a80;
SET_OV7670 + 79 : LUT_DATA = 16'hb084;
SET_OV7670 + 80 : LUT_DATA = 16'hb10c;
SET_OV7670 + 81 : LUT_DATA = 16'hb20e;
SET_OV7670 + 82 : LUT_DATA = 16'hb382;
SET_OV7670 + 83 : LUT_DATA = 16'hb80a;
SET_OV7670 + 84 : LUT_DATA = 16'h4314;
SET_OV7670 + 85 : LUT_DATA = 16'h44f0;
SET_OV7670 + 86 : LUT_DATA = 16'h4534;
SET_OV7670 + 87 : LUT_DATA = 16'h4658;
SET_OV7670 + 88 : LUT_DATA = 16'h4728;
SET_OV7670 + 89 : LUT_DATA = 16'h483a;
SET_OV7670 + 90 : LUT_DATA = 16'h5988;
SET_OV7670 + 91 : LUT_DATA = 16'h5a88;
SET_OV7670 + 92 : LUT_DATA = 16'h5b44;
SET_OV7670 + 93 : LUT_DATA = 16'h5c67;
SET_OV7670 + 94 : LUT_DATA = 16'h5d49;
SET_OV7670 + 95 : LUT_DATA = 16'h5e0e;
SET_OV7670 + 96 : LUT_DATA = 16'h6404;
SET_OV7670 + 97 : LUT_DATA = 16'h6520;
SET_OV7670 + 98 : LUT_DATA = 16'h6605;
SET_OV7670 + 99 : LUT_DATA = 16'h9404;
SET_OV7670 + 100 : LUT_DATA = 16'h9508;
SET_OV7670 + 101 : LUT_DATA = 16'h6c0a;
```

```
SET_OV7670 + 102 : LUT_DATA = 16'h6d55;
SET_OV7670 + 103 : LUT_DATA = 16'h6e11;
SET_OV7670 + 104 : LUT_DATA = 16'h6f9f;
SET_OV7670 + 105 : LUT_DATA = 16'h6a40;
SET_OV7670 + 106 : LUT_DATA = 16'h0140;
SET_OV7670 + 107 : LUT_DATA = 16'h0240;
SET_OV7670 + 108 : LUT_DATA = 16'h13e7;
SET_OV7670 + 109 : LUT_DATA = 16'h1500;
SET_OV7670 + 110 : LUT_DATA = 16'h4f80;
SET_OV7670 + 111 : LUT_DATA = 16'h5080;
SET_OV7670 + 112 : LUT_DATA = 16'h5100;
SET_OV7670 + 113 : LUT_DATA = 16'h5222;
SET_OV7670 + 114 : LUT_DATA = 16'h535e;
SET_OV7670 + 115 : LUT_DATA = 16'h5480;
SET_OV7670 + 116 : LUT_DATA = 16'h589e;
SET_OV7670 + 117 : LUT_DATA = 16'h4108;
SET_OV7670 + 118 : LUT_DATA = 16'h3f00;
SET_OV7670 + 119 : LUT_DATA = 16'h7505;
SET_OV7670 + 120 : LUT_DATA = 16'h76e1;
SET_OV7670 + 121 : LUT_DATA = 16'h4c00;
SET_OV7670 + 122 : LUT_DATA = 16'h7701;
SET_OV7670 + 123 : LUT_DATA = 16'h4b09;
SET_OV7670 + 124 : LUT_DATA = 16'hc9f0;
SET_OV7670 + 125 : LUT_DATA = 16'h4138;
SET_OV7670 + 126 : LUT_DATA = 16'h5640;
SET_OV7670 + 127 : LUT_DATA = 16'h3411;
SET_OV7670 + 128 : LUT_DATA = 16'h3b02;
SET_OV7670 + 129 : LUT_DATA = 16'ha489;
SET_OV7670 + 130 : LUT_DATA = 16'h9600;
SET_OV7670 + 131 : LUT_DATA = 16'h9730;
SET_OV7670 + 132 : LUT_DATA = 16'h9820;
SET_OV7670 + 133 : LUT_DATA = 16'h9930;
SET_OV7670 + 134 : LUT_DATA = 16'h9a84;
SET_OV7670 + 135 : LUT_DATA = 16'h9b29;
SET_OV7670 + 136 : LUT_DATA = 16'h9c03;
SET_OV7670 + 137 : LUT_DATA = 16'h9d4c;
SET_OV7670 + 138 : LUT_DATA = 16'h9e3f;
SET_OV7670 + 139 : LUT_DATA = 16'h7804;
SET_OV7670 + 140 : LUT_DATA = 16'h7901;
SET_OV7670 + 141 : LUT_DATA = 16'hc8f0;
SET_OV7670 + 142 : LUT_DATA = 16'h790f;
SET_OV7670 + 143 : LUT_DATA = 16'hc800;
```

```
SET_OV7670 + 144 : LUT_DATA = 16'h7910;
SET_OV7670 + 145 : LUT_DATA = 16'hc87e;
SET_OV7670 + 146 : LUT_DATA = 16'h790a;
SET_OV7670 + 147 : LUT_DATA = 16'hc880;
SET_OV7670 + 148 : LUT_DATA = 16'h790b;
SET_OV7670 + 149 : LUT_DATA = 16'hc801;
SET_OV7670 + 150 : LUT_DATA = 16'h790c;
SET_OV7670 + 151 : LUT_DATA = 16'hc80f;
SET_OV7670 + 152 : LUT_DATA = 16'h790d;
SET_OV7670 + 153 : LUT_DATA = 16'hc820;
SET_OV7670 + 154 : LUT_DATA = 16'h7909;
SET_OV7670 + 155 : LUT_DATA = 16'hc880;
SET_OV7670 + 156 : LUT_DATA = 16'h7902;
SET_OV7670 + 157 : LUT_DATA = 16'hc8c0;
SET_OV7670 + 158 : LUT_DATA = 16'h7903;
SET_OV7670 + 159 : LUT_DATA = 16'hc840;
SET_OV7670 + 160 : LUT_DATA = 16'h7905;
SET_OV7670 + 161 : LUT_DATA = 16'hc830;
SET_OV7670 + 162 : LUT_DATA = 16'h7926;
SET_OV7670 + 163 : LUT_DATA = 16'h0903;
SET_OV7670 + 164 : LUT_DATA = 16'h3b42;

default : LUT_DATA = 0;
endcase
end
endmodule
```

Module Description :

A low-level SCCB (I2C-compatible) bus master. It implements the physical layer of the protocol, including START/STOP condition generation, bit-serial data transmission, and Acknowledge (ACK) bit monitoring.

Module : debounce (debounce.v)

Module code :

```

////////////////////////////////////
/
// Module Name: debounce
// Description: Module for filtering mechanical switch bounce.
//
//          It ensures that the output signal 'o' becomes high only after
//          the input signal 'i' has been stable at logic high for a
//          continuous duration defined by a 24-bit counter.
////////////////////////////////////
/

module debounce(
    input clk,          // สัญญาณนาฬิกาหลัก (ในโปรเจกต์นี้ใช้ 50MHz จาก Clock Wizard)
    input i,            // สัญญาณอินพุตจากปุ่มกด (ปุ่มที่อาจมีการสั่นของหน้าสัมผัส)
    output reg o        // สัญญาณเอาต์พุตที่สะอาดและเสถียรแล้ว
);

// Register Definitions
reg [23:0] c;          // รีจิสเตอร์ตัวนับขนาด 24 บิต เพื่อสร้างช่วงเวลาหน่วง (Delay)

// Initial State Configuration
initial c = 24'b0;    // กำหนดค่าเริ่มต้นของตัวนับเป็น 0

// Debounce Logic
always @(posedge clk) begin
    if (i == 1) begin
        // หากตรวจพบว่าการกดปุ่ม (In=1) ให้เริ่มทำการนับ
        if (c == 24'hFFFFFF)
            // เมื่อตัวนับทำงานจนถึงค่าสูงสุด (Terminal Count)
            // แสดงว่าสัญญาณอินพุตเสถียรที่สถานะ High นานพอแล้ว
            o <= 1;
        else
            // หากยังนับไม่ถึงเป้าหมาย ให้สถานะเอาต์พุตเป็น Low ต่อไป
            o <= 0;

        // เพิ่มค่าตัวนับในทุกรอบสัญญาณนาฬิกา
        c <= c + 1;
    end
    else begin
        // หากปุ่มถูกปล่อย (In=0) หรือเกิดการสั่นจนสัญญาณหลุดเป็น 0

```

```
// ให้รีเซ็ตตัวนับและเอาต์พุตกลับไปเป็นค่าเริ่มต้นทันที
c <= 24'b0;
o <= 0;

end

end

endmodule
```

Module Description :

A signal conditioning module that filters mechanical contact bounce from the onboard push buttons. It uses a counter-based delay to ensure a stable, glitch-free trigger signal for system resets or camera re-configuration.

Challenge Faced

During the development of this project, our team encountered several significant technical challenges:

1. Edge Visual Noise and Garbage Pixels

- Challenge: When displaying the camera feed on the VGA monitor, noticeable visual noise and striped lines appeared at the edges of the image. This was caused by timing mismatches between the camera's HREF and PCLK signals, which introduced garbage pixels just before or after a valid frame line.
- Solution: Since this hardware limitation could not be fixed directly at the sensor level, we implemented a Digital Cropping and Scaling technique within the VGA controller. We adjusted the coordinate calculation (scaling the image using a 15/16 ratio and shifting the X-axis via the `safe_cam_x` offset) to hide the corrupted pixels at the borders, resulting in a clean and smooth display.

2. Clock Domain Crossing (CDC) Issues

- Challenge: The system inherently relies on two asynchronous clock domains: the camera's pixel clock (PCLK at ~24MHz) for capturing data, and the system's VGA clock (clk25 at 25MHz) for reading data to the display. Fetching data across these domains risks frame tearing and metastability.
- Solution: We resolved this by utilizing a Simple Dual-Port Block RAM as an elastic buffer. This allowed the camera to write data and the VGA controller to read data independently at their respective clock speeds without data collision.

3. Severe BRAM Resource Constraints

- Challenge: The Basys 3 FPGA has a highly limited internal Block RAM capacity (approximately 1.8 Mbit), which is entirely insufficient to store a standard 640x480 full-frame image.
- Solution: We redesigned the memory architecture to capture a smaller 320x240 frame and downsampled the color format to 12-bit RGB444 to fit perfectly into the BRAM. To fill the VGA

screen, we implemented a real-time Pixel Doubling algorithm ($HCnt \gg 1$ and $VCnt \gg 1$) during the display phase, achieving a full-screen output without requiring any additional memory.

4. Hardware Signal Integrity over Physical Wiring

- Challenge: Connecting the OV7670 camera module to the Basys 3 board using standard jumper wires introduced significant electrical noise and parasitic capacitance. High-frequency signals over unshielded wires occasionally caused camera configuration failures.
- Solution: We mitigated this by drastically reducing the frequency of the SCCB configuration clock to approximately 10 kHz. This lower frequency is highly immune to capacitance issues across the physical wires, ensuring that the initial configuration instructions were transmitted to the camera with 100% stability.

Simulation & Testbenches

1. tb_vga.v

```
`timescale 1ns / 1ps

module tb_vga();

    // ประกาศตัวแปรสัญญาณจำลอง
    reg clk25;
    reg [1:0] sw;
    reg [11:0] frame_pixel;

    // ประกาศสายสัญญาณสำหรับรับค่า Output จากโมดูล
    wire [3:0] vga_red, vga_green, vga_blue;
    wire vga_hsync, vga_vsync;
    wire [9:0] HCnt, VCnt;
    wire [16:0] frame_addr;

    // เรียกใช้งานโมดูล vga_disp ของจริงมาทดสอบ
    vga uut (
        .clk25(clk25),
        .sw(sw),
        .vga_red(vga_red),
        .vga_green(vga_green),
        .vga_blue(vga_blue),
        .vga_hsync(vga_hsync),
        .vga_vsync(vga_vsync),
        .HCnt(HCnt),
        .VCnt(VCnt),
        .frame_addr(frame_addr),
        .frame_pixel(frame_pixel)
    );

    // สร้างสัญญาณนาฬิกา 25 MHz (คาบเวลา = 40 ns -> สลับค่าทุกๆ 20 ns)
    always #20 clk25 = ~clk25;

    // กำหนดพฤติกรรมจำลอง
    initial begin
        // กำหนดค่าเริ่มต้น
        clk25 = 0;
    end
endmodule
```

```

sw = 2'b00;
frame_pixel = 12'hF00; // สมมติว่าอ่านค่าสีแดง (Red) มาจาก RAM ตลอดเวลา

// รอเวลาให้ระบบรันไปประมาณ 2 มิลลิวินาที (เพื่อให้เห็นสัญญาณ HSYNC สลับค่าหลายๆ
รอบ)

#2000000;

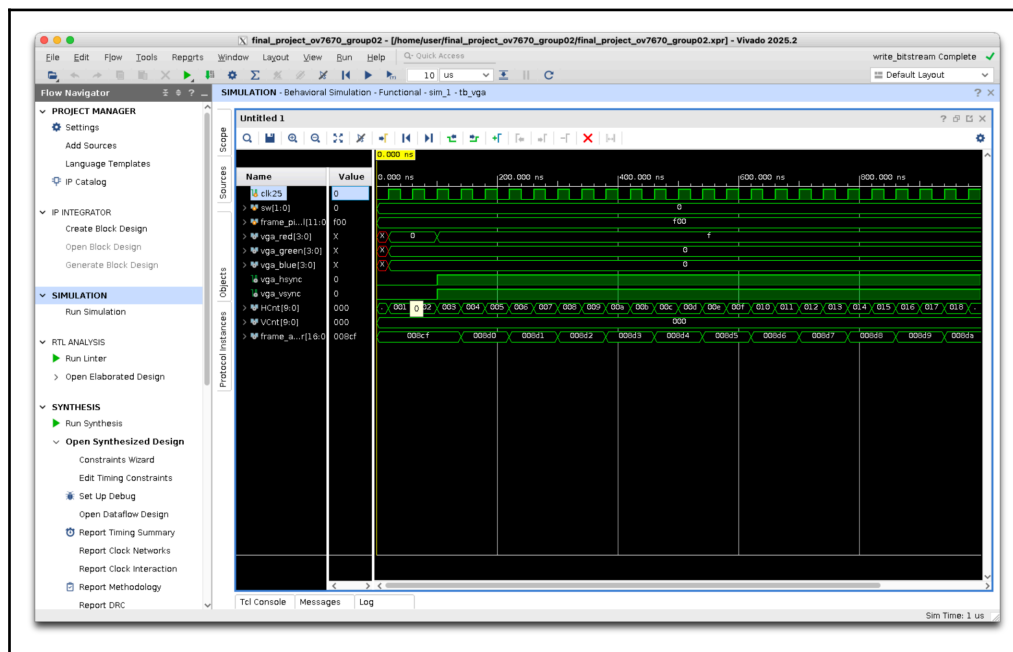
// จบการจำลอง
$finish;

end

endmodule

```

○ Simulation Results: VGA Controller Timing



- Behavioral simulation waveform of the VGA Controller (vga_disp module)
- **Waveform Analysis:** Based on the behavioral simulation waveform of the VGA controller, we can verify the hardware's operational logic as follows:
 - i. System Clock (clk25): The 25 MHz clock is operating correctly, serving as the primary time base for driving the display counters.

-
- ii. Pixel Counters (HCnt and VCnt): * The horizontal counter (HCnt) correctly increments by 1 on every rising edge of the clock (displaying hexadecimal values such as 011, 012, 013...). The vertical counter (VCnt) remains at 000, indicating that the controller is currently scanning the very first horizontal line of the display.
 - iii. Data Routing and Pipeline Delay (frame_pixel to vga_red):
 - In this simulation, the testbench feeds a bright red pixel value from the RAM (frame_pixel = 12'hF00).
 - The output signal vga_red successfully transitions from an unknown state (X) to f, while vga_green and vga_blue remain at 0. This confirms that the color separation and output routing logic functions perfectly.
 - Key Observation: There is a visible delay between the input pixel arrival and the VGA output. This represents the expected Pipeline Synchronization delay (2 clock cycles via the video_on shift registers), ensuring the color data perfectly aligns with the sync signals without timing violations.
 - iv. Synchronization Signals (vga_hsync): The vga_hsync signal transitions from 0 (Low) to 1 (High), conforming to the standard Active-Low operation of VGA timing after the horizontal retrace period ends.
 - v. Memory Addressing and Digital Zoom (frame_addr): The read address (frame_addr) increments sequentially (...008d1, 008d2, 008d3...). Notably, the frame_addr holds the same value for exactly two clock cycles before incrementing. This explicitly verifies our Pixel Doubling (Digital Zoom) logic ($\text{HCnt} \gg 1$), proving that each pixel is fetched and displayed twice horizontally to stretch the 320×240 image across the full 640×480 screen.

2. tb_pixel_filter.v

```
`timescale 1ns / 1ps

module tb_pixel_filter();

    reg clk;
    reg [1:0] sw;
    reg [11:0] pixel_in;
    wire [11:0] pixel_out;

    // เรียกใช้งานโมดูลฟิลเตอร์
    pixel_filter uut (
        .clk(clk),
        .sw(sw),
        .pixel_in(pixel_in),
        .pixel_out(pixel_out)
    );

    always #5 clk = ~clk; // สร้าง Clock จำลอง

    initial begin
        // เริ่มต้น
        clk = 0;
        sw = 2'b00;
        pixel_in = 12'h000;
        #20;
        // -----
        // 1. ทดสอบโหมดปกติ (sw = 00)
        // -----
        sw = 2'b00;
        pixel_in = 12'hF00; #20; // ป้อนสีแดง
        pixel_in = 12'h0F0; #20; // ป้อนสีเขียว
        pixel_in = 12'h00F; #20; // ป้อนสีน้ำเงิน

        // -----
        // 2. ทดสอบโหมด Grayscale (sw = 01)
        // -----
        sw = 2'b01;
        pixel_in = 12'hF00; #20; // สีแดง -> ควรออกเป็นสีเทา (ค่า R=G=B)
        pixel_in = 12'h00F; #20; // สีน้ำเงิน -> ควรออกเป็นสีเทาเข้ม
    end
endmodule
```

```
// -----
// 3. ทดสอบโหมด Color Splash (sw = 10) ดูเฉพาะสีน้ำเงิน
// -----

sw = 2'b10;

pixel_in = 12'hF00; #20; // สีแดง -> ไม่ผ่านเงื่อนไข ควรออกเป็นขาวดำ
pixel_in = 12'h00F; #20; // สีน้ำเงินเด่น -> ควรออกเป็นสีน้ำเงินเหมือนเดิม

// -----
// 4. ทดสอบโหมด Edge Detection (sw = 11)
// -----

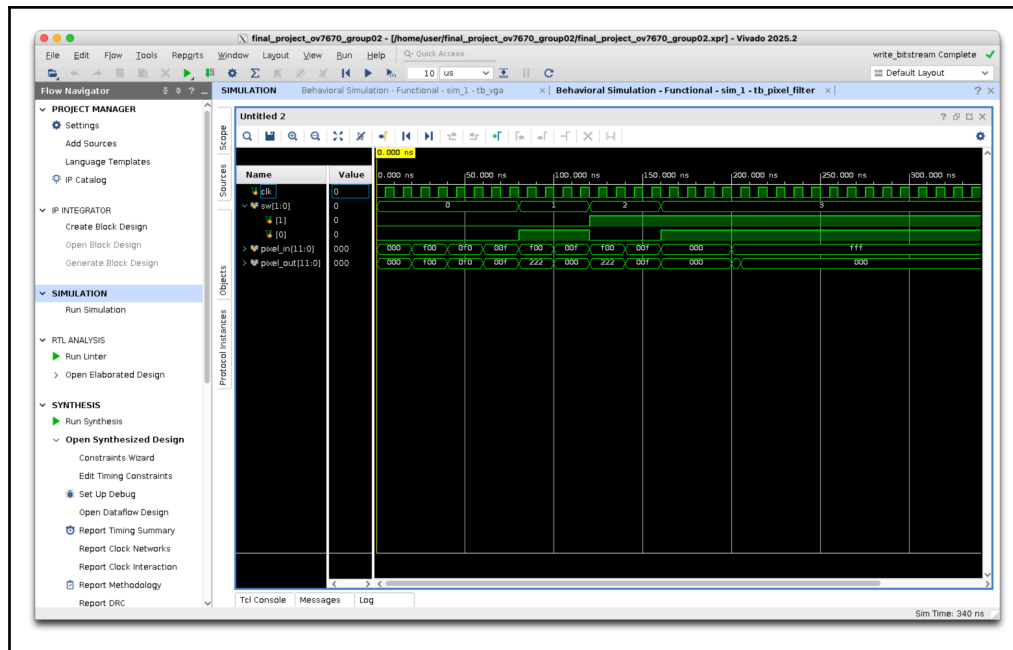
sw = 2'b11;

pixel_in = 12'h000; #20; // ดำต่อเนื่อง
pixel_in = 12'h000; #20; // ไม่มีความต่าง -> ควรเป็นสีดำ
pixel_in = 12'hFFF; #20; // จุกๆ เปลี่ยนเป็นสีขาว! (เกิดขอบ) -> ควรเป็นสีขาว
pixel_in = 12'hFFF; #20; // ขาวต่อเนื่อง ไม่มีความต่าง -> ควรกลับไปเป็นสีดำ

#100;
$finish;

end
endmodule
```

○ Simulation Result: Image Processing Filter Logic



○ Behavioral simulation waveform of the image processing engine (pixel_filter module)

-
- **Waveform Analysis:** The waveform above demonstrates the functional verification of the hardware-based image processing pipeline. The simulation tests all four operational modes controlled by the switch input (sw[1:0]):
 - i. Normal Mode (sw = 0 / 2'b00):
 - The module correctly receives raw input colors (e.g., Red 12'hf00, Green 12'h0f0, Blue 12'h00f) and passes them through the color correction and gamma adjustment logic, outputting the expected stabilized color values.
 - ii. Grayscale Filter (sw = 1 / 2'b01):
 - When a pure red pixel (12'hf00) is injected, the output immediately transitions to a uniform gray value (12'h222). This proves that the luminance calculation equation ($Y \approx 0.3R + 0.6G + 0.1B$) is functioning correctly in hardware.
 - iii. Color Splash - Blue Focus (sw = 2 / 2'b10):
 - The logic correctly isolates the blue color channel. When a red pixel (12'hf00) is input, it is suppressed into grayscale (12'h222). However, when a blue pixel (12'h00f) is input, it satisfies the threshold condition and is allowed to output as its original blue color (12'h00f).
 - iv. 1D Edge Detection (sw = 3 / 2'b11):
 - No Edge: When the input pixel data remains constant (e.g., a continuous stream of 12'h000 or 12'hfff), the output correctly stays at black (12'h000), indicating no structural difference between adjacent pixels.
 - Edge Detected: At the simulation mark just past 200ns, the input abruptly transitions from black (12'h000) to white (12'hfff). The absolute difference calculation detects this sharp contrast, causing the

output to spike to white (12'hfff). This confirms that the real-time edge detection algorithm successfully identifies object boundaries.

- **Conclusion:** The simulation confirms that both the combinational multiplexing logic and the sequential pixel-delay registers in the pixel_filter module operate flawlessly across all requested visual effects without timing violations.

3. tb_capture.py

```
import os
from pathlib import Path

import cocotb
from cocotb.triggers import Timer, FallingEdge
from cocotb.clock import Clock
from cocotb_tools.runner import get_runner

@cocotb.test()
async def test_camera_capture(dut):
    """Testbench สำหรับจำลองการรับค่าจากกล้อง OV7670 (RGB565 -> RGB444)"""

    cocotb.start_soon(Clock(dut.pclk, 40, units="ns").start())

    dut.vsync.value = 0
    dut.href.value = 0
    dut.d.value = 0

    dut._log.info("System Reset Complete. Waiting for VSYNC...")
    await Timer(100, units="ns")

    await FallingEdge(dut.pclk)
    dut.vsync.value = 1
    await Timer(120, units="ns")
    await FallingEdge(dut.pclk)
    dut.vsync.value = 0

    # 4. จำลองการรับพิกเซลต่อเนื่อง (Pipeline Data Stream)
    dut.href.value = 1

    # --- เริ่มคล็อกที่ 1 ---
    dut._log.info(">>> Sending Pixel 1: RED (0xF800)")
    dut.d.value = 0xF8 # ส่ง High Byte 1
    await FallingEdge(dut.pclk)

    # --- เริ่มคล็อกที่ 2 ---
    dut.d.value = 0x00 # ส่ง Low Byte 1
    await FallingEdge(dut.pclk)

    # --- เริ่มคล็อกที่ 3 ---
```

```

dut._log.info(">>> Sending Pixel 2: GREEN (0x07E0)")
dut.d.value = 0x07 # ส่ง High Byte 2 (ส่งต่อเนื่องเลย)
await FallingEdge(dut.pclk)

# ณ จุดนี้ (คล็อกที่ 3) ลอจิกประมวลผล Pixel 1 เสร็จแล้ว! ตรวจสอบค่าตอบได้เลย
dout_val = int(dut.dout.value)
dut._log.info(f"[Check Pixel 1] DOUT: {hex(dout_val)}, WE: {dut.we.value}")
assert dut.we.value == 1, "WE should be 1 for Pixel 1"
assert dout_val == 0xf00, f"Expected 0xf00 but got {hex(dout_val)}"

# --- เริ่มคล็อกที่ 4 ---
dut.d.value = 0xE0 # ส่ง Low Byte 2
await FallingEdge(dut.pclk)

# --- เริ่มคล็อกที่ 5 ---
dut.href.value = 0 # จบการส่งเฟรม เอา HREF ลง
await FallingEdge(dut.pclk)

# ณ จุดนี้ (คล็อกที่ 5) ลอจิกประมวลผล Pixel 2 เสร็จแล้ว!
dout_val = int(dut.dout.value)
dut._log.info(f"[Check Pixel 2] DOUT: {hex(dout_val)}, WE: {dut.we.value}")
assert dut.we.value == 1, "WE should be 1 for Pixel 2"
assert dout_val == 0x0f0, f"Expected 0x0f0 but got {hex(dout_val)}"

await Timer(100, units="ns")
dut._log.info("Camera Capture Simulation Complete! Check the Waveform.")

def runner():
    top_module = "ov7670_capture"
    test_module = "tb_capture"
    sim = os.getenv("SIM", "icarus")
    proj_path = Path(__file__).resolve().parent
    sources = [proj_path / "camera_capture.v"]

    runner = get_runner(sim)
    runner.build(
        sources=sources,
        hdl_toplevel=top_module,
        always=True,

```

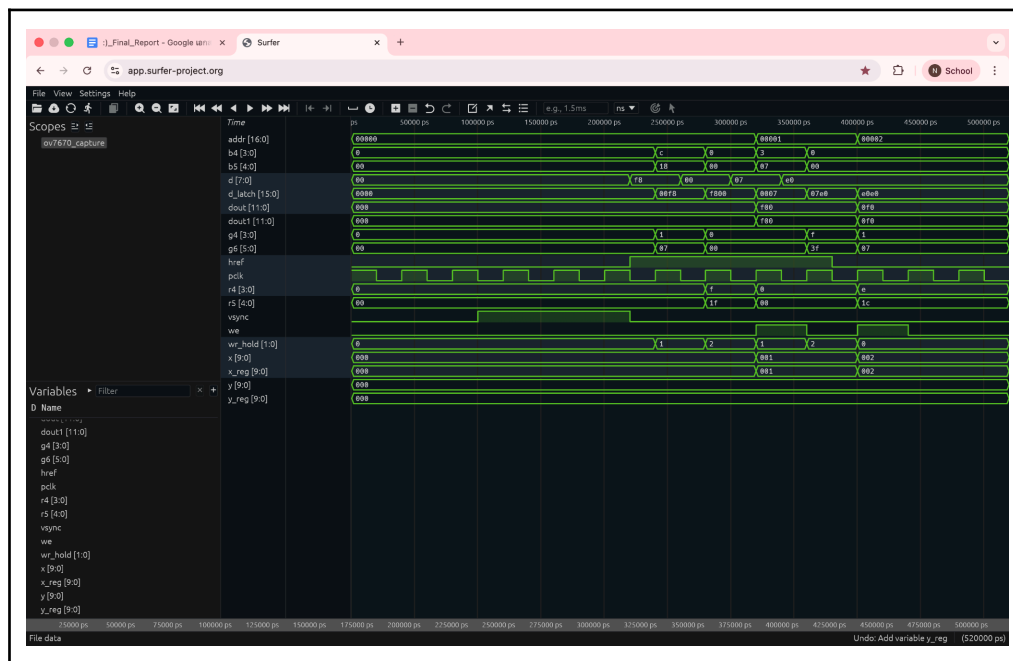
```

waves=True,
timescale=("1ns", "1ps"),
)
runner.test(
    hdl_toplevel=top_module,
    test_module=test_module,
    waves=True,
)

if __name__ == "__main__":
    os.chdir(os.path.dirname(os.path.abspath(__file__)))
    runner()

```

○ Simulation Result: OV7670 Camera Capture Pipeline



- Behavioral co-simulation waveform of the ov7670_capture module using Cocotb
- **Waveform Analysis:** This simulation verifies the hardware logic responsible for interfacing with the OV7670 camera, handling the synchronization signals, and compressing the incoming pixel data. The timeline demonstrates several key behaviors:
 - i. Frame Synchronization (vsync & href): The simulation begins with a vsync pulse (HIGH then LOW), which

correctly signals the start of a new video frame and resets the internal address and coordinate counters (addr, x, y to 0). Shortly after, href transitions to HIGH, indicating the start of a valid horizontal video line where active pixel data is being transmitted.

- ii. **Byte Latching and Assembly (pclk, d[7:0], d_latch[15:0]):** The camera transmits a 16-bit RGB565 pixel across two consecutive clock cycles (1 byte at a time). For the first pixel, the waveform shows d[7:0] receiving the high byte 0xF8 on the first pclk cycle, followed by the low byte 0x00 on the second cycle. The d_latch register successfully concatenates them into a complete 16-bit word (0xF800, representing bright Red).
- iii. **Format Conversion and Data Output (dout[11:0]):** Due to the BRAM constraints on the Basys 3 board, the 16-bit RGB565 data must be downsampled. The logic correctly truncates the least significant bits of each color channel in real-time. As seen in the waveform, the input 0xF800 (Red) is perfectly converted to a 12-bit RGB444 format 0xF00. Similarly, the second pixel 0x07E0 (Green) is accurately converted to 0x0F0.
- iv. **Memory Write Control (wr_hold & we):** The wr_hold state machine carefully tracks the incoming byte sequence. The Write Enable (we) signal asserts (goes HIGH) exactly once every two clock cycles—only when a full 16-bit pixel has been fully assembled and converted. This ensures that the BRAM only stores valid, complete pixel data.
- v. **Address Generation (addr, x):** Immediately following a successful memory write (we assertion), the memory address (addr) and the horizontal pixel coordinate (x) correctly increment by 1 (e.g., from 00000 to 00001, and then to 00002), confirming the accurate sequential memory mapping for the frame buffer.